

DAT410 - Design of AI Systems

Group 69, Module 3		
Teammates	Jana Leong	Chung Fung Oi
Course	Bachelor of Computer Science	
Email	janale@student.chalmers.se	funco@student.chalmers.se
No. of hours spent	15	

We hereby declare that we have both actively participated in solving every exercise. All solutions are entirely our own work, without having taken part in other solutions.

1. Reading and reflection

- Read the paper [Hidden Technical Debt in Machine Learning Systems Links to an external site.](#) and write an (approximately) half-page summary of its contents and your take-aways from it. *(Since many of you have asked, this is half a page including both summary and take-aways)*

The paper posits that although machine learning systems can be developed and deployed rapidly, maintaining them over time is disproportionately difficult and expensive due to the accumulation of system-level technical debt. This debt is largely driven by the CACE principle (Changing Anything Changes Everything), which highlights that no input signal is truly independent, making isolated improvements nearly impossible. In practice, ML code only makes for a tiny fraction of the overall infrastructure, while the majority consists of “glue code” and “pipeline jungles”, which significantly increase fragility and maintenance overhead. Additionally, ML systems are particularly vulnerable to feedback loops and unstable data dependencies, which can lead to improvement deadlocks, hidden coupling between components, and failures that are difficult to detect or diagnose.

Key takeaways from the paper stress the importance of prioritizing system robustness over marginal accuracy gains. Seemingly harmless additions can impose long-term costs that hinder future development and experimentation. To mitigate these risks, there is a need for comprehensive monitoring, including tracking prediction bias to ensure alignment between training data and real-world behavior, and enforcing action limits as safeguards for systems that interact directly with the world. Maintaining a healthy codebase also requires deliberate simplification: engineers should routinely remove unnecessary features through evaluations and reduce dependency lock-in by encapsulating black-box ML packages behind stable, shared APIs. Beyond technical practices, the paper advocates for a cultural shift toward cross-functional teams that value maintainability, reproducibility, monitoring, and system clarity as much as algorithmic performance improvements.

Ultimately, the paper reframes ML engineering as a long-term systems engineering challenge rather than a purely modeling task. It warns that small gains in predictive accuracy are often outweighed by the complexity they introduce, and that unmanaged technical debt can silently erode a system’s capacity to evolve—even when models appear to be functioning correctly.

2. Implementation

- The file [Cities.zip](#) contains a dataset of weather, climate and pollution measurements from five Chinese cities. The set given here is based on a [UCI dataset Links to an external site](#), but restricted to measurements at 12 noon and observations with no missing data.
 - The target variable (label) is called "PM_HIGH" and represents the event that the pollution level, measured as a concentration of particles called PM2.5, is higher than 100 $\mu\text{g}/\text{m}^3$.
 - The features (covariates) are described [here](#).
- **Implement, as a class in Python, a classifier** using the **fit, predict, score** pattern (this pattern is familiar from scikit-learn and was described in the lecture).
 - For training, the classifier should use k-means clustering to identify cluster centroids. Each centroid should be assigned a label equal to the *majority label of points in the cluster*.
 - At test time, to predict the label for a new point, first infer the cluster assignment of the new point using the trained k-means centroids. Then, predict the label corresponding to the centroid of the inferred cluster.
 - The fit, predict and score methods themselves should be of your own implementation but *may use* an existing k-means clustering implementation (e.g., from scikit-learn) inside them.
- **Apply your classifier** to the Chinese city pollution dataset, by predicting the binary target variable PM_HIGH from remaining variables.
 - **Train and validate** your system on the cities Beijing and Shenyang. Finally, **evaluate** your classifier separately on the cities Guangzhou and Shanghai.
- Submit a PDF report briefly summarizing your implementation and describing in detail your evaluation of the system: How well did the system work? For example, what was the training accuracy, what was the validation accuracy (on Beijing and Shenyang)? What about the performance of the second pair of cities (Guangzhou and Shanghai)?
 - Make sure to submit your Python **code**.

We implemented a prototype based classifier following the fit-predict-score pattern, similar to scikit-learn.

Training (fit)

- K-means clustering is applied to the feature vectors X (excluding PM_HIGH and CITY)
- Each training point belongs to exactly one cluster
- For each cluster, we examine the true labels of points assigned to it
- The cluster centroid is assigned a label equal to the majority label of its points (0 or 1)

Prediction (predict)

- For a new data point, we find the nearest centroid using the train k-means model

- The predicted label is the label assigned to that centroid
- This means that the classifier predicts “You belong to cluster c; most training points in c has label y_c , so that is the prediction.”

Scoring (score)

- Accuracy is computed as the fraction of correctly predicted labels.

Setup

City-based split:

- Training + validation cities: Beijing and Shenyang
- Evaluation cities: Guangzhou and Shanghai
- This ensures that the final test results measure generalization to unseen cities and not just interpolation.

Validation split:

- Beijing and Shenyang split was: 80% training and 20% validation (stratified by PM_HIGH)
- Validation is used only to select the number of k clusters

Features

- All meteorological and pollution-related variables except:
 - PM_HIGH because this is the target
 - CITY (removed to avoid leakage)

Results

Validation accuracy by k:

k= 5 → 0.725

k=10 → 0.777

k=20 → 0.781

k=30 → 0.788

k=40 → 0.793

k=50 → 0.796

k=60 → 0.807

k=70 → 0.794

k=100 → 0.779

Best k: 60

On the validation set (Beijing and Shenyang), the validation accuracy of peaks at k = 60 (0.807). This means that our clusters are becoming more specific & helps predictions within those cities. Thus for these cities it is better to use k = 60 rather than k = 50.

On the test set (Guangzhou and Shanghai) however, the test accuracy drops when $k = 60$, $0.721 < 0.731$ ($k = 50$). So despite $k = 60$ being better previously, $k = 60$ actually generalizes worse on unseen cities.

k = 60	k = 50
Final test accuracy: 0.7214206437291898 Train accuracy: 0.7754749568221071 Test accuracy: 0.7214206437291898 Overall test accuracy: 0.7214206437291898 Per-city test accuracy: CITY Score Guangzhou 0.766272 Shanghai 0.676536 Train accuracy: 0.7754749568221071 Trainval accuracy: 0.7816925734024179 Balanced test accuracy: 0.5195029 Confusion Matrix: [[1889 595] [158 61]]	Final test accuracy: 0.7306696263411024 Train accuracy: 0.7737478411053541 Test accuracy: 0.7306696263411024 Overall test accuracy: 0.7306696263411024 Per-city test accuracy: CITY Score Guangzhou 0.781065 Shanghai 0.680237 Train accuracy: 0.7737478411053541 Trainval accuracy: 0.7789291882556131 Balanced test accuracy: 0.5141260 Confusion Matrix: [[1919 565] [163 56]]

Although $k=60$ achieved the highest validation accuracy on Beijing and Shenyang, it performed slightly worse on unseen cities. This suggests that larger k values capture city-specific patterns that do not generalize well. Moderate k values (around 40-50) provide a better trade-off between fit and cross-city robustness. This is realistic and important to our findings.

Thus we can choose to select $k = 50$ in order to do our predictions as we are predicting for other cities. We can also see that Guangzhou generalizes better than Shanghai, suggesting that Shanghai has pollution-weather relationships that differ more strongly from Beijing/ Shenyang.

Balancing accuracy and class imbalance

Because $PM_HIGH = 1$ is the minority class, we also computed balanced accuracy, defined as the average recall (true positive cases) across both classes. See [here](#) for classification related metrics we used.

Balanced accuracy test = 0.514

This means that the model is close to random guessing and indicates a high recall for $PM_HIGH = 0$ and very low recall for $PM_HIGH = 1$.

The confusion matrix above can explain and confirm this behavior:

```
[[1919 565]
 [ 163 56]]
```

- The model correctly predicts most non-high-pollution cases but missing the majority of high-pollution cases. This is also a known limitation of majority-vote clustering method on

imbalance data.

Overall, the classifier achieves reasonable accuracy (~73%) when evaluated on unseen cities. This performance is substantially above the majority-class baseline. However, balanced accuracy is low, showing poor detection of high-pollution cases, where the system generalizes moderately well across cities but is biased toward the majority class. This is also one of the main challenges of unsupervised prototype-based methods under class imbalance and distribution shift.

Naturally an extension would be to replace hard majority labelling with distance-weighted voting over multiple nearest centroids, which could possibly improve recall for rare events without changing the k-means framework.

3. Discussion

- In most cases, the accuracy of a system when deployed is lower than the accuracy measured during development (training). Was this the case for your system? What are possible reasons for this? Discuss which assumptions are made during development that may not hold, and what could be done to improve the system. Do these assumptions hold for the problem above?

Our system exhibited a clear performance gap between development and deployment. During development, when training on data from Beijing and Shenyang, the system achieved a relatively high validation accuracy, peaking at 0.807 (at $k = 60$). However, when evaluated on other real world data (Guangzhou and Shanghai), the test accuracy decreased to approximately 0.731 (at $k = 50$) and 0.721 (at $k = 60$). This shows that the results observed during development were more optimistic than the system's real-world performance.

A possible reason for this performance drop may be due to the assumption that the data used for training is similar to the data used after deployment. During training, the model assumes that weather conditions affect pollution in approximately the same way across all cities. In reality, this assumption does not fully hold. Pollution patterns can be influenced by city-specific factors such as climate, geography, sources of pollution, and city layouts. This effect is reflected in the per-city test results, where the model performs better for data from Guangzhou than in Shanghai. This suggests that the patterns learned during training can generalize better to more cities than others.

Model complexity could also be a factor contributing to the dip in performance. Increasing the number of clusters improves validation accuracy for training as the model captures more detailed patterns in the data. However, these detailed patterns may be more specific to the cities used during training, and not generalize well to other cities with different environmental conditions. As a result, using too many clusters can reduce performance after deployment. This explains why $k = 60$, even though it performs best during validation, works worse than smaller values such as $k = 50$ for other cities. This behavior is similar to overfitting, where a model performs well on training data but poorly on new data.

Another important issue is class imbalance. In this problem, the model predicts one of two classes: low pollution ($PM_HIGH = 0$) or high pollution ($PM_HIGH = 1$). High pollution events occur much less frequently than low pollution events, which leads to an imbalanced dataset. During development, performance is mainly evaluated using overall accuracy, which assumes that both classes are equally important. As a result, the model can possibly achieve relatively high accuracy by predicting most low-pollution cases while failing to identify as many high-pollution events. This issue is clearly shown by the balanced accuracy and the confusion matrix, which indicate that the model has low recall for high-pollution cases. This limitation is common in methods that rely on majority voting when the data is imbalanced.

Overall, the lower performance observed after deployment is expected because several assumptions made during development do not fully hold in real-world conditions. The system

assumes that pollution–weather relationships are consistent across cities, that clusters learned from training data generalize well to new environments, and that overall accuracy is a sufficient measure of performance. In practice, these assumptions are violated due to differences between cities and strong class imbalance in the data.

To improve the system, future work could involve training on data from a larger and more diverse set of cities to capture a wider range of pollution patterns. The model could also be improved by modifying how cluster labels are assigned, for example by using distance-weighted voting instead of assigning each cluster a single majority label. In addition, placing greater emphasis on evaluation metrics that account for class imbalance could help guide the development of models that better detect rare but important high-pollution events.

module3group69

February 10, 2026

```
[1]: import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import accuracy_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

class KMeansMajorityClassifier:
    """
    Classifier:
        - fit(): run k-means on X, then assign each cluster a label = majority y
        ↪ in that cluster
        - predict(): assign each new x to nearest centroid, output that cluster's
        ↪ label
        - score(): accuracy on (X, y)

    Notes:
        - Works with pandas DataFrames
        - One-hot encodes any non-numeric columns (e.g., CITY), and aligns
        ↪ columns at test time
        - Median-imputes missing values (though your data should have none) and
        ↪ standardizes features
    """
    def __init__(self, n_clusters=10, random_state=0, n_init="auto"):
        self.n_clusters = n_clusters
        self.random_state = random_state
        self.n_init = n_init

    def _prepare_X(self, X, fit=False):
        # Accept DataFrame or array-like
        if isinstance(X, pd.DataFrame):
            X_df = X.copy()
        else:
            X_df = pd.DataFrame(X)

        # One-hot encode any non-numeric columns
```

```

        cat_cols = [c for c in X_df.columns if not pd.api.types.
↪is_numeric_dtype(X_df[c])]
        X_oh = pd.get_dummies(X_df, columns=cat_cols, drop_first=False)

        if fit:
            self.feature_columns_ = X_oh.columns
        else:
            # Align test columns to training columns
            X_oh = X_oh.reindex(columns=self.feature_columns_, fill_value=0)

        # Impute + scale
        if fit:
            self.imputer_ = SimpleImputer(strategy="median")
            X_imp = self.imputer_.fit_transform(X_oh)
            self.scaler_ = StandardScaler()
            X_scaled = self.scaler_.fit_transform(X_imp)
        else:
            X_imp = self.imputer_.transform(X_oh)
            X_scaled = self.scaler_.transform(X_imp)

        return X_scaled

    def fit(self, X, y):
        y_arr = np.asarray(y).astype(int)
        X_scaled = self._prepare_X(X, fit=True)

        self.kmeans_ = KMeans(
            n_clusters=self.n_clusters,
            random_state=self.random_state,
            n_init=self.n_init
        )
        self.kmeans_.fit(X_scaled)

        clusters = self.kmeans_.labels_
        self.cluster_label_ = {}

        for k in range(self.n_clusters):
            idx = np.where(clusters == k)[0]
            if len(idx) == 0:
                self.cluster_label_[k] = 0
            else:
                labels = y_arr[idx]
                # Majority vote; tie breaks to 1 if mean >= 0.5
                self.cluster_label_[k] = int(labels.mean() >= 0.5)

        return self

```

```

def predict(self, X):
    X_scaled = self._prepare_X(X, fit=False)
    cluster_ids = self.kmeans_.predict(X_scaled)
    return np.array([self.cluster_label_.get(c, 0) for c in cluster_ids],
dtype=int)

def score(self, X, y):
    y_arr = np.asarray(y).astype(int)
    return accuracy_score(y_arr, self.predict(X))

```

```

[2]: import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score

paths = [
    "/content/Beijing_labeled.csv",
    "/content/Shenyang_labeled.csv",
    "/content/Guangzhou_labeled.csv",
    "/content/Shanghai_labeled.csv",
    "/content/Chengdu_labeled.csv"
]

dfs = []
for p in paths:
    df = pd.read_csv(p)
    df["CITY"] = os.path.basename(p).split("_")[0]
    dfs.append(df)

data = pd.concat(dfs, ignore_index=True)

# Train and validate on Beijing & Shenyang
train_cities = ["Beijing", "Shenyang"]
# Evaluate on Guangzhou and Shanghai
test_cities = ["Guangzhou", "Shanghai"]

trainval_df = data[data["CITY"].isin(train_cities)]
test_df = data[data["CITY"].isin(test_cities)]
train_df, val_df = train_test_split(
    trainval_df,
    test_size=0.2,
    random_state=42,
    stratify=trainval_df["PM_HIGH"]
)

```

```

X_train = train_df.drop(columns=["PM_HIGH", "CITY"])
y_train = train_df["PM_HIGH"]

X_val = val_df.drop(columns=["PM_HIGH", "CITY"])
y_val = val_df["PM_HIGH"]

X_test = test_df.drop(columns=["PM_HIGH", "CITY"])
y_test = test_df["PM_HIGH"]

ks = [5, 10, 20, 30, 40, 50]
val_scores = {}

for k in ks:
    clf = KMeansMajorityClassifier(n_clusters=k, random_state=42)
    clf.fit(X_train, y_train)
    val_scores[k] = clf.score(X_val, y_val)

print("Validation accuracy by k:")
for k, acc in val_scores.items():
    print(f"k={k:2d} → {acc:.3f}")

best_k = max(val_scores, key=val_scores.get)
print("Best k:", best_k)

X_trainval = trainval_df.drop(columns=["PM_HIGH", "CITY"])
y_trainval = trainval_df["PM_HIGH"]

clf = KMeansMajorityClassifier(n_clusters=best_k, random_state=42)
clf.fit(X_trainval, y_trainval)

y_pred = clf.predict(X_test)

print("Final test accuracy:", accuracy_score(y_test, y_pred))

print("Train accuracy:", clf.score(X_train, y_train))
print("Test accuracy:", clf.score(X_test, y_test))

# Predict once
y_pred = clf.predict(X_test)

# Overall accuracy
overall_acc = accuracy_score(y_test, y_pred)
print("Overall test accuracy:", overall_acc)

# Per-city accuracy (use test_df for CITY)
results = pd.DataFrame({

```

```

        "CITY": test_df["CITY"].values,
        "y_true": y_test.values,
        "y_pred": y_pred
    })

    per_city_acc = results.groupby("CITY").apply(
        lambda g: accuracy_score(g["y_true"], g["y_pred"])
    )

    print("Per-city test accuracy:")
    print(per_city_acc)

    print("Train accuracy:", clf.score(X_train, y_train))
    print("Trainval accuracy:", clf.score(X_trainval, y_trainval))

```

Validation accuracy by k:

k= 5 → 0.725

k=10 → 0.777

k=20 → 0.781

k=30 → 0.788

k=40 → 0.793

k=50 → 0.796

Best k: 50

Final test accuracy: 0.7306696263411024

Train accuracy: 0.7737478411053541

Test accuracy: 0.7306696263411024

Overall test accuracy: 0.7306696263411024

Per-city test accuracy:

CITY

Guangzhou 0.781065

Shanghai 0.680237

dtype: float64

Train accuracy: 0.7737478411053541

Trainval accuracy: 0.7789291882556131

/tmp/ipython-input-925352746.py:89: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
per_city_acc = results.groupby("CITY").apply(
```

```
[3]: from sklearn.metrics import balanced_accuracy_score
```

```

bal_acc = balanced_accuracy_score(y_test, y_pred)
print("Balanced test accuracy:", bal_acc)

```

Balanced test accuracy: 0.5141260229854632

```
[4]: from sklearn.metrics import confusion_matrix  
      print(confusion_matrix(y_test, y_pred))
```

```
[[1919  565]  
 [ 163   56]]
```

```
[ ]:
```